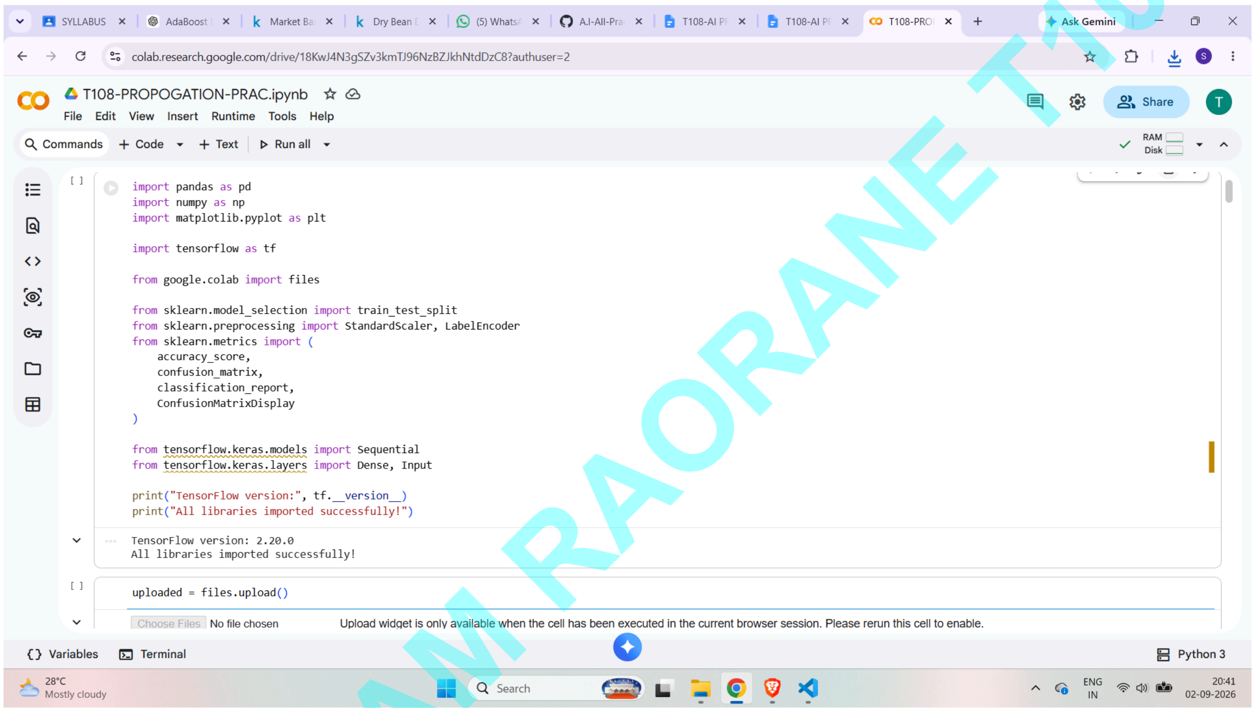
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

AIM : Feed Forward Backpropagation Neural Network
Implement the Feed Forward Backpropagation algorithm to train a neural network.
Use a given dataset to train the neural network for a specific task.
Evaluate the performance of the trained network on test data.

IMPORT LIBRARIES



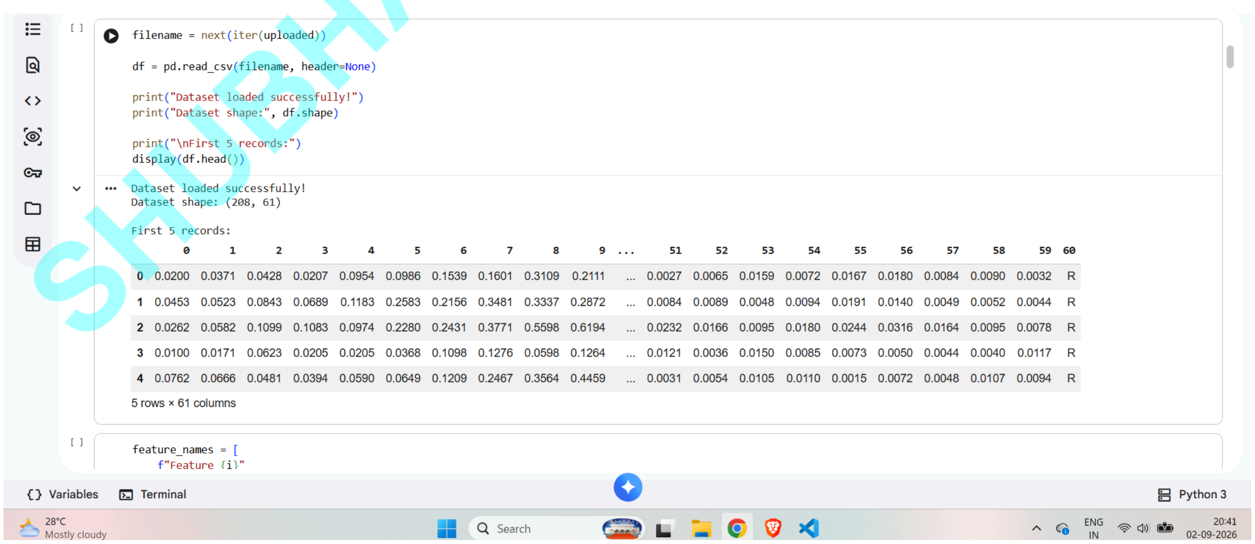
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

print("TensorFlow version:", tf.__version__)
print("All libraries imported successfully!")
```

```
TensorFlow version: 2.20.0
All libraries imported successfully!
```

```python
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

READ CSV



```python
filename = next(iter(uploaded))

df = pd.read_csv(filename, header=None)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

print("\nFirst 5 records:")
display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (208, 61)

First 5 records:
```

	0	1	2	3	4	5	6	7	8	9	...	51	52	53	54	55	56	57	58	59	60
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027	0.0065	0.0159	0.0072	0.0167	0.0180	0.0084	0.0090	0.0032	R
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084	0.0089	0.0048	0.0094	0.0191	0.0140	0.0049	0.0052	0.0044	R
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232	0.0166	0.0095	0.0180	0.0244	0.0316	0.0164	0.0095	0.0078	R
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121	0.0036	0.0150	0.0085	0.0073	0.0050	0.0044	0.0040	0.0117	R
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031	0.0054	0.0105	0.0110	0.0015	0.0072	0.0048	0.0107	0.0094	R

5 rows × 61 columns

```python
feature_names = [
    f"Feature {i}"
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

FEATURE NAMES

```
feature_names = [
    f"Feature_{i}"
    for i in range(1, 61)
]

df.columns = feature_names + ["Target"]

print("Column names added successfully!")

display(df.head())
```

```
Column names added successfully!
```

	Feature_1	Feature_2	Feature_3	Feature_4	Feature_5	Feature_6	Feature_7	Feature_8	Feature_9	Feature_10	...	Feature_52	Feature_53	Feature_54	Feature_55	Feature_5
0	0.0200	0.0371	0.0428	0.0207	0.0954	0.0986	0.1539	0.1601	0.3109	0.2111	...	0.0027	0.0065	0.0159	0.0072	0.016
1	0.0453	0.0523	0.0843	0.0689	0.1183	0.2583	0.2156	0.3481	0.3337	0.2872	...	0.0084	0.0089	0.0048	0.0094	0.019
2	0.0262	0.0582	0.1099	0.1083	0.0974	0.2280	0.2431	0.3771	0.5598	0.6194	...	0.0232	0.0166	0.0095	0.0180	0.024
3	0.0100	0.0171	0.0623	0.0205	0.0205	0.0368	0.1098	0.1276	0.0598	0.1264	...	0.0121	0.0036	0.0150	0.0085	0.007
4	0.0762	0.0666	0.0481	0.0394	0.0590	0.0649	0.1209	0.2467	0.3564	0.4459	...	0.0031	0.0054	0.0105	0.0110	0.001

5 rows × 61 columns

```
print("Dataset Information:")
df.info()
```

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:42 02-09-2026

DATA SET INFORMATION , MISSING VALUES, TARGET CLASSES

```
print("Dataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum().sum())

print("\nTarget Classes:")
print(df["Target"].unique())
```

```
 11  Feature_12  208 non-null    float64
 12  Feature_13  208 non-null    float64
 13  Feature_14  208 non-null    float64
 14  Feature_15  208 non-null    float64
 15  Feature_16  208 non-null    float64
 16  Feature_17  208 non-null    float64
 17  Feature_18  208 non-null    float64
 18  Feature_19  208 non-null    float64
 19  Feature_20  208 non-null    float64
 20  Feature_21  208 non-null    float64
 21  Feature_22  208 non-null    float64
 22  Feature_23  208 non-null    float64
 23  Feature_24  208 non-null    float64
 24  Feature_25  208 non-null    float64
 25  Feature_26  208 non-null    float64
 26  Feature_27  208 non-null    float64
 27  Feature_28  208 non-null    float64
 28  Feature_29  208 non-null    float64
 29  Feature_30  208 non-null    float64
 30  Feature_31  208 non-null    float64
 31  Feature_32  208 non-null    float64
 32  Feature_33  208 non-null    float64
```

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:42 02-09-2026

```
X = df.drop("Target", axis=1)
y = df["Target"]

print("Number of input features:", X.shape[1])
print("Number of target values:", y.shape[0])
```

```
Number of input features: 60
Number of target values: 208
```

```
encoder = LabelEncoder()

y = encoder.fit_transform(y)

print("Original classes:", encoder.classes_)
print("Encoded classes:", np.unique(y))
```

```
Original classes: ['M' 'R']
Encoded classes: [0 1]
```

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:42 02-09-2026

SHUBHAM RAORANE T108

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

TRAIN TEST, TRAINING SAMPLES, TESTING SAMPLES

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", x_train.shape[0])
print("Testing samples:", x_test.shape[0])
```

```
Training samples: 166
Testing samples: 42
```

```
scaler = StandardScaler()

x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

print("Feature scaling completed.")
```

```
Feature scaling completed.
```

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:43 02-09-2026

MODEL SUMMARY AND MODEL CREATED

```
model = Sequential([

    Input(shape=(x_train.shape[1],)),

    Dense(32, activation="relu"),

    Dense(16, activation="relu"),

    Dense(1, activation="sigmoid")
])

print("Neural network created successfully!")
```

```
Neural network created successfully!
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	1,952
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17

Total params: 2,497 (9.75 KB)
Trainable params: 2,497 (9.75 KB)
Non-trainable params: 0 (0.00 B)

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:44 02-09-2026

MODEL COMPILE

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

Variables Terminal Python 3

28°C Mostly cloudy Search ENG IN 20:44 02-09-2026

SHUBHAM RAORANE T108

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

TEST LOSS, ACCURACY

```python
test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\n===== TEST PERFORMANCE =====")

print("Test Loss:", test_loss)

print("Test Accuracy:", test_accuracy)

print(
    "Test Accuracy Percentage:",
    test_accuracy * 100,
    "%"
)
```

```
===== TEST PERFORMANCE =====
Test Loss: 0.5997765064239502
Test Accuracy: 0.7857142686843872
Test Accuracy Percentage: 78.57142686843872 %
```

PROBABILITIES , ACCURACY%

```python
probabilities = model.predict(X_test)

y_pred = (
    probabilities >= 0.5
).astype(int).flatten()

print("Predictions generated successfully!")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 113ms/step
Predictions generated successfully!
```

```python
accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

```
Accuracy: 0.7857142857142857
Accuracy Percentage: 78.57142857142857 %
```

CLASSIFICATION REPORT

```python
print("\n===== CLASSIFICATION REPORT =====")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Mine", "Rock"]
    )
)
```

```
===== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

        Mine       0.74      0.91      0.82        22
        Rock       0.87      0.65      0.74        20

    accuracy                           0.79        42
   macro avg       0.80      0.78      0.78        42
weighted avg       0.80      0.79      0.78        42
```


SHUBHAM RAORANE T108

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

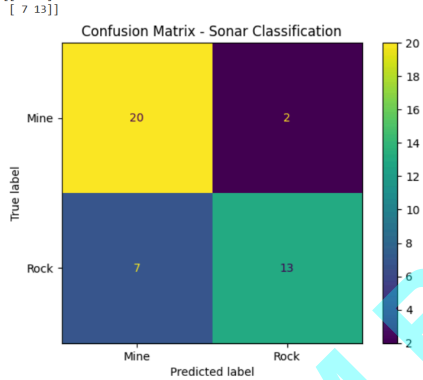
SUBJECT : AI

CONFUSION MATRIX

```python
cm = confusion_matrix(
    y_test,
    y_pred
)

print("\n===== CONFUSION MATRIX =====")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Mine", "Rock"]
)

disp.plot()

plt.title("Confusion Matrix - Sonar Classification")
plt.show()
```

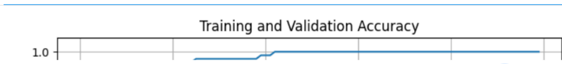
```
===== CONFUSION MATRIX =====
[[20  2]
 [ 7 13]]
```

```
===== CONFUSION MATRIX =====
[[20  2]
 [ 7 13]]
```



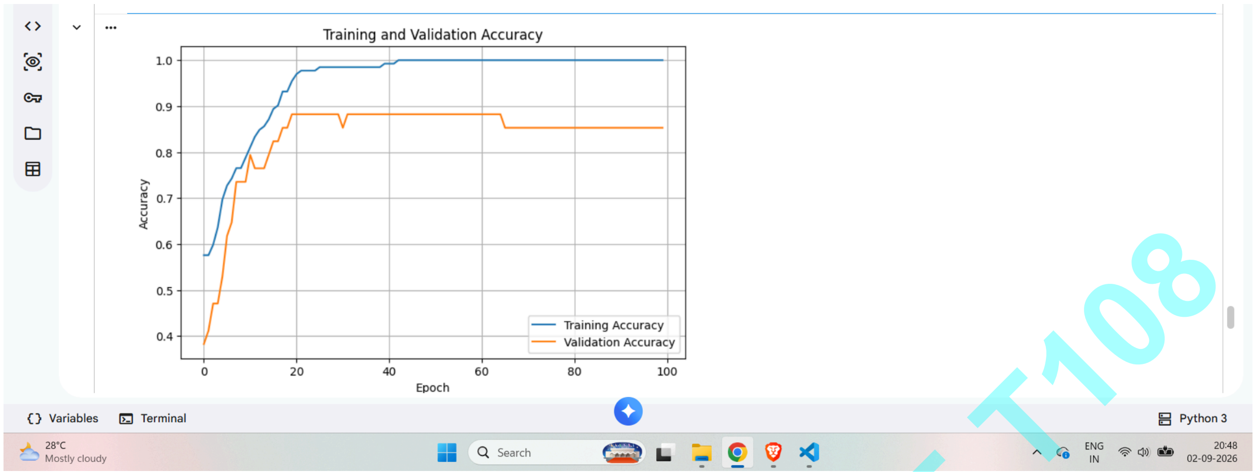
TRAINING ACCURACY AND VALIDATION GRAPH

```python
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.title("Training and Validation Accuracy")

plt.legend()
plt.grid()

plt.show()
```



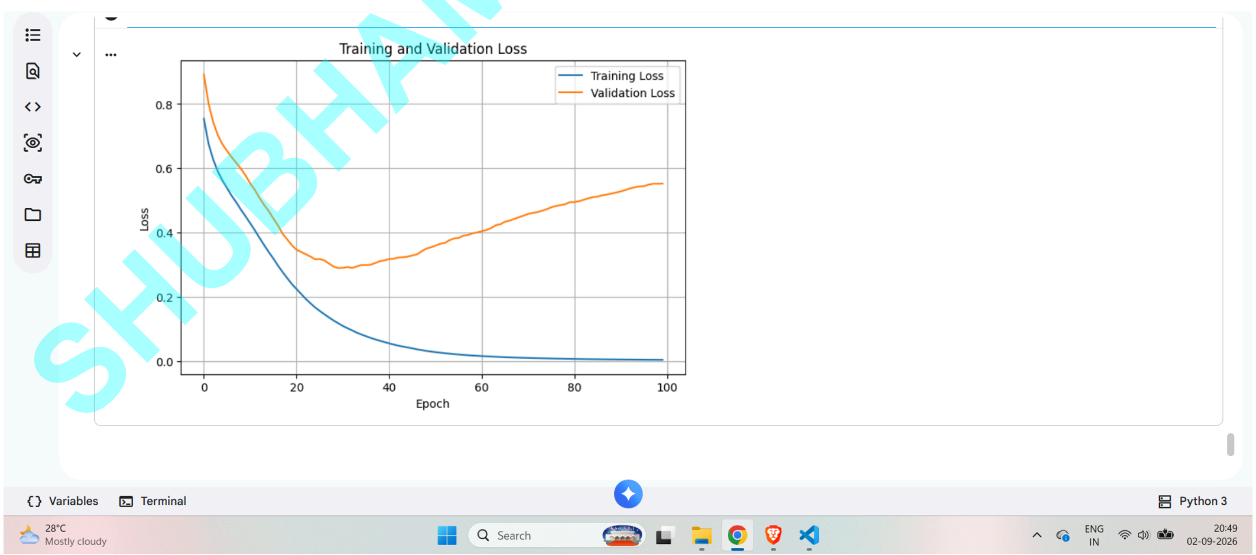
SHUBHAM RAORANE T108

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI



TRAINING AND VALIDATION LOSS

```python
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title("Training and Validation Loss")

plt.legend()
plt.grid()

plt.show()
```



SHUBHAM RAORANE T108